This work has been submitted to the IEEE for possible publication.
Copyright may be transferred without notice, after which this version may no longer be accessible.

arXiv:2406.13511v2 [cs.DC] 9 Mar 2025

Slice-Level Scheduling for High-Throughput and Load-Balanced LLM Serving

Ke Cheng, Wen Hu, Zhi Wang, Hongen Peng,
Jianguo Li *Senior Member,IEEE,* Sheng Zhang *Senior Member,IEEE*

***Abstract*—Large language models (LLMs) iteratively generate text token by token, with memory usage increasing with the length of generated token sequences. Since the request generation length is generally unpredictable, it is difficult to estimate the time and memory required to process requests, thus posing a challenge for effective request scheduling. Conventional sequence-level scheduling (SLS) serves requests in a first-come first-served (FCFS) manner with static batching where requests with short generation lengths are delayed until those with long ones have finished generation. Besides, to avoid out-of-memory (OOM) errors, SLS batches requests using a small batch size, which limits throughput. Recently proposed iteration-level scheduling (ILS) improves this with continuous batching, timely completing requests and dynamically adding new ones, but often limits the number of parallel-processing requests to OOM errors, thus compromising throughput. Moreover, both SLS and ILS fail to effectively balance workload across multiple LLM instances. To tackle these challenges, we propose slice-level scheduling (SCLS). By splitting the predefined maximal generation length limit into slices and serving batches slice by slice, it provides a precise range of serving time and memory usage for batched requests, laying the foundation for effective scheduling. Experiments confirm that compared with SLS and ILS schedulers, SCLS can improve throughput by up to 315.8% and greatly mitigate load imbalance with proposed batching and offloading algorithms.**

***Index Terms*—Large language model, machine learning system, transformer serving, adaptive batching, load balancing.**

I. Introduction

LARGE language models (LLMs) are currently playing an important role in a wide range of applications, especially interactive web chatbots, such as CodeFuse [1] developed by Ant Group. In addition to the daily chat capability, CodeFuse is also widely acclaimed for its enhanced ability to generate, optimize, and interpret various programming languages, including SQL, Java, Python, and C++, boasting a peak concurrent usage with thousands of online users. Therefore, an efficient serving system with high throughput and scalability is essential to handle such a heavy workload.

To achieve high throughput for LLM serving, it is significant to fully utilize the parallel computing capability of GPUs with batch serving. Static batching and continuous batching are two batch-serving techniques for LLM inference. Static batching pads the raw input of batched requests to the same length before serving and iteratively generates tokens until all requests generate the end-of-sequence (EOS) token or the predefined iteration number limit is reached. Continuous batching leverages customized CUDA kernels to remove request padding and enable dynamic exits and joins of requests during batch serving.

However, the time and GPU memory required for serving a request increases with the generation length (i.e., the number of generated tokens) because of the autoregressive generation pattern and key-value cache mechanism of LLM inference. Since the request generation length is generally unpredictable, it is difficult to estimate the serving time and memory consumption of requests in advance. Therefore, it is challenging to schedule requests to take full advantage of these batch-serving techniques for high throughput.

Existing deep learning serving systems [2], [3] adopt the sequence-level scheduling (SLS) to batch requests in a first-come first-served (FCFS) manner with a fixed batch size and serve them with static batching as shown in Fig. 1a, which leads to severe computational inefficiencies. Firstly, for static batching, in a batch, requests with short inputs are padded, and requests with short outputs have to wait for those with long outputs to finish before they are returned together. During the waiting, completed requests still participate in the computation and generate invalid tokens. Secondly, without knowing request generation lengths, the SLS scheduler directly sets the iteration number limit for static batching to the predefined maximal generation length and defaults to a small batch size to prevent out-of-memory (OOM) errors, thus failing to fully exploit the GPU's parallel computing capability and compromising throughput.

To eliminate the request padding and waiting caused by SLS and improve computational efficiency, iteration-level scheduling (ILS) is introduced in recent studies [4]–[6]. Current ILS techniques leverage continuous batching to timely return completed requests and add newly arrived requests for processing in an FCFS manner at each iteration, as shown in Fig. 1b. However, without knowing request generation lengths, many ILS techniques [4], [6] adopt conservative memory management strategies that limit the number of parallel-processing requests to avoid OOM errors, thus under-utilizing the GPU and hurting throughput.

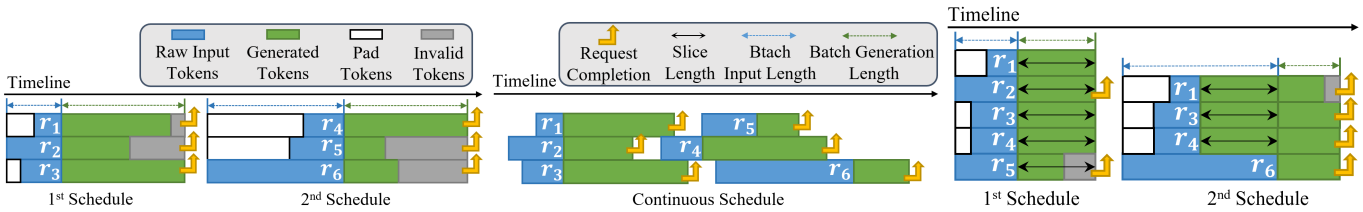
Moreover, existing SLS and ILS schedulers [2], [3], [6] leverage the round-robin policy to offload requests to multiple LLM instances, which causes unbalanced distribution of computational overhead and memory consumption across LLM instances due to the variance of request generation lengths.

In this paper, we propose slice-level scheduling (SCLS) to address the aforementioned challenges from a pure scheduling perspective. The core idea is to split the predefined maximal generation length limit into fixed-length slices and

Ke Cheng and Sheng Zhang are with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China (e-mail: ketonmi@outlook.com, sheng@nju.edu.cn).

Wen Hu, Zhi Wang, Hongen Peng, and Jianguo Li are with Ant Group, Hangzhou 310063, China (e-mail: {huwen.hu, wangchun.wz, hongen.phe, lijg.zero}@antgroup.com).

This work was done during Ke Cheng's internship at Ant Group.



(a) Sequence-level scheduling (SLS). (b) Iteration-level scheduling (ILS). (c) Slice-level scheduling (SCLS).

Fig. 1: Illustration of various scheduling techniques, where r_i represents the ith arrived request.

serve batches slice by slice. Completed requests are returned promptly, while incomplete requests are rescheduled with newly arrived requests until finished. Generation slicing can enhance both static and continuous batching. We focus on strengthening static batching in this paper. In Section VII, we discuss promising technique solutions for applying the idea of generation slicing to continuous batching, which we leave as our future work.

Compared with the maximal generation length limit, the small slice length results in a narrow range of generation length at each schedule, thus providing a precise range of memory consumption and serving time for batches. With the precise range of memory consumption, requests can be adaptively batched together with as large a batch size as possible to improve the throughput without exceeding the GPU memory as shown in Fig. 1c. With the precise range of serving time, batches can be scheduled to LLM instances using the max-min policy [7] to achieve load-balancing. All in all, generation slicing lays the foundation for effective scheduling. Besides, since requests with long outputs are rare in real-world applications such as ChatGPT [8] and CodeFuse, most requests can be returned in a few slices without waiting for long.

We implement SCLS as the gateway to route requests and conduct extensive experiments with LLM instances deployed in the production environment of Ant Group using CodeFuse request traces. Experimental results show that SCLS can achieve a high request throughput and a more balanced workload across LLM instances. Our main contribution is listed as follows.

- We confirm the inefficiency and load imbalance of SLS and ILS schedulers through experiments. Besides, we find that requests with long outputs are rare by analyzing the ShareGPT dataset and request traces collected from CodeFuse's logs. These motivate the birth of SCLS.
- We propose SCLS. By limiting the number of iterations to a small slice length in each schedule, it controls the serving time and memory usage of batches to a precise range, laying the foundation for effective scheduling.
- We design a dynamic programming-based batching algorithm and a max-min-based offloading algorithm for SCLS. They divide requests into batches to minimize the batch serving time and balance the computational workload across multiple LLM instances, respectively.
- We conduct extensive experiments to confirm the superiority of SCLS. Equipped with various inference engines, SCLS can improve request throughput by up to 315.8%, reduce response time by up to 91.1%, and greatly mitigate load imbalance compared with SLS and ILS schedulers.

II. Preliminaries

LLMs generate text by predicting the next token for an input token sequence and adding the predicted token to the input iteratively until the EOS token is generated. Such a generation pattern is called **auto-regressive generation**. To prevent the generation process from being too long, a maximal generation length limit is always set. When this limit is reached, even if the EOS token has not been generated, the generated results are returned to users. The **request input length** is the number of tokens in the raw request text, while the **request generation length** is the number of generated tokens.

In static batching, requests are padded to the same length and processed together. Therefore, the **batch input length** is the length of the longest request in the batch. The **batch generation length** is the total number of iterations for the batch, depending on the longest request generation length and the iteration number limit.

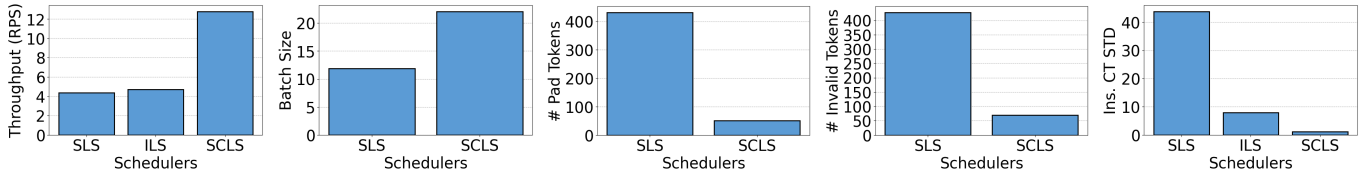
To accelerate LLM inference, the key and value tensors produced by transformer blocks of LLMs are cached for reuse. In the first iteration of the auto-regressive generation, the token sequence of raw request text is fed to the LLM, and the key and value tensors of these input tokens are computed and cached. In subsequent iterations, only the latest generated token is fed to the LLM for computation and cache of key and value tensors. Thus, the generation process can be split into two phases. The first iteration is the computationally heavy **prefill phase**. Subsequent iterations are the computationally light **decoding phase**.

III. Motivation

A. Inefficient Serving of SLS and ILS

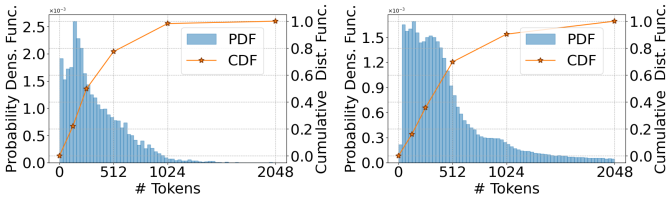
We conduct experiments to confirm the inefficient serving of SLS and ILS schedulers. We implement the SLS and SCLS schedulers using deepspeed-inference [9] as the inference engine and leverage deepspeed-fastgen [6] as the ILS scheduler. In the experiment, 8 LLaMA2-13B [10] instances are deployed on 8 NVIDIA A100 80GB GPUs, and requests with various input and generation lengths arrive at a rate of 20 per second. Other settings are the same as in Section V. The experimental results are shown in Fig. 2.

From Fig. 2a, we can see that the throughput of SLS is much smaller than that of SCLS. This is because SLS serves requests with a fixed small batch size. Moreover, SLS simply batches requests in an FCFS manner, which leads to a lot of pad tokens and generated invalid tokens caused by the variance of the batched requests' input lengths and generation lengths. Besides, although continuous batching can eliminate



(a) Throughput. (b) Batch size. (c) #Pad token. (d) #Invalid token. (e) Load imbalance.

Fig. 2: Inefficiency and load imbalance of SLS and ILS schedulers. Sub-figures (a) and (e) show that SCLS can significantly outperform SLS and ILS schedulers in terms of throughput and load balancing. Sub-figures (b)˜(d) present that SCLS's superior performance comes from the increase of batch size and decrease of the number of pad tokens and invalid tokens.



(a) CodeFuse. (b) ShareGPT.

Fig. 3: Probability density function (PDF) and cumulative distribution function (CDF) of the request generation length in actual scenarios.

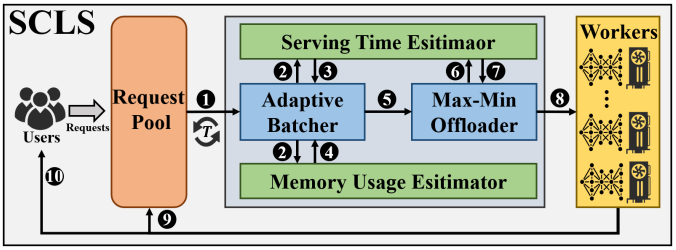


Fig. 4: SCLS system overview. The arrows represent data transferred between modules, where ❶ is requests, ❷ is temporary batches, ❸ and ❼ are estimated serving time, ❹ is estimated memory consumption, ❺, ❻, and ❽ are batches, ❾ is uncompleted requests, and ❿ is completed requests.

pad tokens and invalid tokens in the serving process, the throughput of ILS is also much lower than that of SCLS, because the ILS scheduler adopts a conservative memory management mechanism that limits the number of parallel-processing requests.

B. Unbalanced Workload of SLS and ILS

We record the time when each LLM instance completes serving at the end of the experiment. Fig. 2e presents the standard deviation (STD) of the completion time (CT) of LLM instances. Since both SLS and ILS schedulers use the round-robin policy to offload requests, requests with long and short generation lengths may be assigned to different instances, resulting in unbalanced memory consumption and computational overhead across instances. For the SLS scheduler, instances that have more requests with long generation lengths take more time to serve requests. For the ILS scheduler, instances that have more requests with long generation lengths consume more memory and cannot promptly add new requests for processing due to insufficient memory, causing a long request queuing time. Such a load imbalance accumulates to result in a large variance in the CT of LLM instances. Hence, the CT STD for LLM instances under SLS and ILS schedulers is much larger than that of SCLS.

C. Real-World Generation Length Distribution

To understand the generation length distribution in real-world LLM-based web applications, we analyze 400,000 pieces of data from ShareGPT[1] and one-month request traces collected from CodeFuse's logs. We depict the distribution of request input and generation lengths in Fig. 3, where we can find that the vast majority of requests have a small generation length of less than 512 in both CodeFuse and ShareGPT data. Hence, if we split the maximal generation length limit (e.g., 2048) into slices with a small slice length (e.g., 128) and serve batched requests slice by slice, most requests can be returned in time within a small number of slices. In addition, since the iteration number limit is set to the small slice length, we can get a precise range of memory consumption and batch together as many requests as possible while ensuring that OOM errors will not occur, thus increasing throughput.

Moreover, for static batching, we can also get a precise range of batch serving time. Hence, load balancing can be achieved by offloading batches with the max-min policy where the batch with the longest serving time is offloaded to the least loaded instance.

Considering the above, we propose SCLS, which splits the predefined maximal generation length limit into fixed-length slices, limits the number of iterations in each batch serving to the slice length, and serves requests slice by slice. Although requests with long generation lengths suffer from the extra overhead of recomputing the prefill phase at every reschedule, the overhead is acceptable because these long-generated requests are rare as shown in Fig. 3.

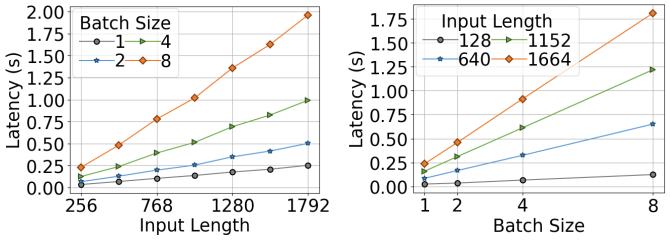
IV. Solution Description

A. System Overview

SCLS has four core components, a serving time estimator, a memory usage estimator, an adaptive batcher, and a max-min offloader. The overview of SCLS is presented in Fig. 4.

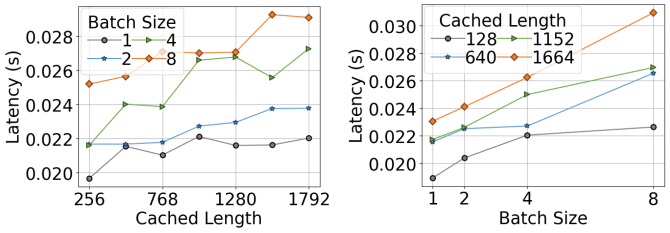
SCLS periodically fetches all requests from the request pool with a time interval T and hands them over to the adaptive batcher. The adaptive batcher leverages a dynamic

[1]ShareGPT [11] is a publicly available website that shares conversations from users chatting with ChatGPT [8].



(a) $T_{prefill}(N, L_i)$ under different input length. (b) $T_{prefill}(N, L_i)$ under different batch sizes.

Fig. 5: Prefill latency under deepspeed-inference (DS).



(a) $\tau_{decode}(l, N)$ under different context lengths. (b) $\tau_{decode}(l, N)$ under different batch sizes.

Fig. 6: Per-iteration decoding latency under DS.

programming-based algorithm to group the requests into batches with the goal of minimizing the total serving time. In each step of the batching algorithm, the adaptive batcher leverages the estimated serving time of temporarily grouped batches to update the state transfer function and utilizes estimated memory consumption to guarantee that the serving process will not cause OOM errors. After the batching procedure finishes, batches will be offloaded to workers by the max-min offloader. After batch serving is finished, completed requests are returned to the user, and uncompleted requests are sent to the request pool to be scheduled with newly arrived requests.

A worker in Fig. 4 is an LLM instance running on a specific inference engine. The worker's receiving thread receives the batch from the offloader and saves it in a local queue, while its processing thread continuously fetches a batch from the queue and serves it. In this paper, we implement the HF-worker and DS-worker to load and run LLM instances using huggingface transformers (HF) [12] and deepspeed-inference (DS) [9] as inference engines, respectively.

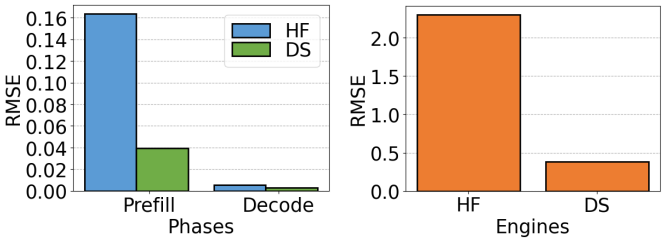
B. Efficient Serving Time Estimation

Inference engines, such as HF and DS, serve requests with static batching, where the batch serving time is determined by the batch size, batch input length, and batch generation length.

Given a batch $\mathcal{B}$ whose batch size, batch input length, and batch generation length are separately represented as N, L_i, and L_o. The batch serving time is calculated by

$$T_{serve}(N, L_i, L_o) = T_{prefill}(N, L_i) + T_{decode}(N, L_i, L_o), \quad (1)$$

where $T_{prefill}(N, L_i)$, and $T_{decode}(N, L_i, L_o)$ are the serving latency in the prefill and decoding phase, respectively. Since the prefill phase computes key and value tensors for all the input tokens, the latency is determined by the batch input length L_i and the batch size N.



(a) Error of 1 iteration. (b) Error of 128 iterations.

Fig. 7: Serving time estimation error of various inference engines, where HF and DS are short for huggingface-transformers and deepspeed-inference, respectively.

In the decoding phase, the serving procedure runs iteratively. Therefore, $T_{decode}(N, L_i, L_o)$ can be calculated by summing up the per-iteration decoding latency, which is expressed by

$$T_{decode}(N, L_i, L_o) = \sum_{l=L_i+1}^{Li+L_o} \tau_{decode}(l, N), \quad (2)$$

where $\tau_{decode}(l, N)$ denotes the per-iteration latency. In each decoding iteration, for each request, only one token is fed to the LLM and extracts features from all the previously cached tokens to make a prediction, and hence $\tau_{decode}(l, N)$ is determined by the context length l and the batch size N.

We profile the prefill latency $T_{prefill}(N, L_i)$ of an LLaMA2-13B instance using DS as the inference engine under various batch sizes and batch input lengths. To clearly show the individual effect of batch size N and batch input length L_i on $T_{prefill}(N, L_i)$, we plot two sub-figures with these two factors as the horizontal axis in Fig. 5. Similarly, we also profile the per-iteration latency $\tau_{decode}(l, N)$ and show the effect of context length l and batch size N in Fig. 6.

For the prefill latency $T_{prefill}(N, L_i)$, we can observe that when the batch size is fixed, $T_{prefill}(N, L_i)$ increases linearly with the batch input length L_i from Fig. 5a. Besides, as shown in Fig. 5b, when the input length is fixed, the $T_{prefill}(N, L_i)$ increases linearly with the batch size N as well. For HF, we observe the same phenomenon. Therefore, we use the following function to fit $T_{prefill}(N, L_i)$.

$$T_{prefill}(N, L_i) = p_1 \cdot N \cdot L_i + p_2 \cdot N + p_3 \cdot L_i + p_4, \quad (3)$$

where p_1, p_2, p_3, and p_4 are parameters to be fitted. Under this function, $T_{prefill}(N, L_i)$ is a linear function of the batch input length L_i when the batch size N is fixed. Besides, when L_i is fixed, $T_{prefill}(N, L_i)$ is also a linear function of N, which is consistent with our observations. We leverage the profiled prefill latency data of various engines to fit the function. Fig. 7a depicts the fitting error metric, root-mean-square error (RMSE), which represents the average gap between the estimated and actual latency. As shown in Fig. 7a, the average gap is merely 0.16s for HF and less than 0.04s for DS, achieving an accurate estimation.

For the per-iteration decoding latency $\tau_{decode}(l, N)$. we can observe from Fig. 6a that when the batch size is fixed, $\tau_{decode}(l, N)$ rises with fluctuations as the context length increases. Besides, as shown in Fig. 6b, when the context length is fixed, $\tau_{decode}(l, N)$ also shows a rising trend with the

increase of the batch size, and when the context length is large, the tendency tends to be linear. For HF, we also observe the same phenomenon. Hence, we try to estimate the per-iteration latency the same way we estimate the prefill latency with

$$\tau_{decode}(l, N) = d_1 \cdot N \cdot l + d_2 \cdot N + d_3 \cdot l + d_4, \quad (4)$$

where d_1, d_2, d_3, and d_4 are parameters to be fitted. Fig. 7a presents the estimation error of $\tau_{decode}(l, N)$ for various inference engines. We can find that the fitting error is negligible, which confirms that the estimation is accurate.

By incorporating Eq. (4) into Eq. (2), we can obtain the estimated serving time of the decoding phase. Adding the decoding latency to the prefill latency computed by Eq. (3), we can get an estimate of the batch serving time. We also evaluate the estimation error of overall batch serving time for an iteration number of 128 under various batch input lengths and batch sizes. As shown in Fig. 7b, the average error is merely 2.3s for HF and less than 0.4s for DS, which confirms that the estimation is accurate.

From Fig. 7, we can find that both the per-iteration error and multi-iteration error of HF are much higher than those of DS. This is because DS leverages customized CUDA kernels to accelerate the computation of transformer blocks, so its latency bases are much smaller than that of HF, and hence its average estimation error is smaller than that of HF. Moreover, since the prefill latency is typically much larger than the per-iteration decoding latency, the average estimation error of the prefill latency is also larger than that of the decoding latency.

SCLS sets the iteration number limit for static batching to a small slice length S. When the number of iterations reaches S, the batch serving ends regardless of whether or not all requests have generated the EOS token. Therefore, we can consider the batch generation length L_o to be S for batch serving. Hence, the batch serving time of SCLS can be accurately and quickly estimated by $T_{serve}(N, L_i, S)$, where N and L_i can be directly obtained before batch serving.

If all requests in the batch generate the EOS token before the generation length reaches S, the batch serving will finish prematurely and the estimated serving time will be inaccurate. However, since S is always set small, the likelihood of the premature finish happening is small, and the performance of SLCS won't be affected. We verify this in Section V.

The serving time estimator is trained on offline profiled data and then applied for online prediction. Compared with exhaustively profiling the serving time under various batch input lengths and batch sizes for a given slice length, such an analytical model can be established merely by profiling the per-iteration prefill and decoding latency, greatly reducing the profiling overhead. Furthermore, the online estimation is also simple and efficient.

C. Practical Memory Usage Estimation

During LLM serving, GPU memory is mainly occupied by the parameters of the LLM and the key-value cache of requests. The memory consumed by the key-value cache is determined by the batch size, batch input length, and batch generation length. Since for static batching, both the pad tokens and generated invalid tokens produce key-value cache, the memory consumed by key-value cache is calculated by

$$M_{kv}(N, L_i, L_o) = (L_i + L_o) \cdot N \cdot \Delta, \quad (5)$$

where N, L_i, and L_o respectively represent the batch size, the batch input length, and the batch generation length. Δ denotes the per-token memory usage of key and value tensors, which is determined by the model architecture.

To avoid OOM errors during serving, the memory usage of the key-value cache can not exceed the maximal available GPU memory M_{ava}, which can be calculated by

$$M_{ava} = M_{cap} - M_{model} - M_{engine}, \quad (6)$$

where M_{cap} is the GPU memory capacity, M_{model} is the memory consumed by LLM parameters, and M_{engine} is the memory utilized by the inference engine to store engine-specific data.

Theoretically, given the batch size N, the batch input length L_i, and the slice length S, in SCLS, the memory estimator can determine whether the OOM errors will occur during batch serving by simply checking if the following constraint hold

$$M_{kv}(N, L_i, S) \leq M_{ava}. \quad (7)$$

In addition, by incorporating Eq. (5) into Eq. (7) and rearranging it, we can obtain Eq. (8), which indicates that given the batch input length L_i and slice length S, limiting the batch size to be less than N_{max} can avoid OOM errors.

$$N_{max} = \lfloor \frac{M_{ava}}{\Delta \cdot (L_i + S)} \rfloor. \quad (8)$$

In practice, there is a gap between the actual GPU memory usage and the theoretical usage for different inference engines due to the engine-specific memory management mechanism.

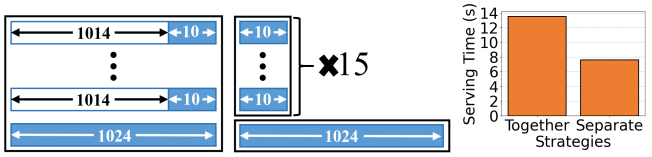
HF is an inference engine based entirely on pytorch [13]. It produces memory fragmentation that cannot be utilized during the batch serving. Therefore, for HF, it is necessary to multiply M_{ava} in Eq. (7) by an additional coefficient ζ less than 1, and the constraint is rewritten as

$$M_{kv}(N, L_i, S) \leq \zeta \cdot M_{ava}. \quad (9)$$

DS has an inflexible memory management mechanism. We experimentally find that it is unable to determine whether OOM errors will occur using Eq. (9) for DS, even if ζ is set to a small value. Therefore, for DS, for a given slice length S, we profile the maximal batch size for various batch input lengths and make rules for determining whether OOM will occur during batch serving.

D. Serving Time-Oriented Batching

As shown in Fig. 8, under a slice length of 128, given 15 requests with an input length of 10 and 1 request with an input length of 1024, if they are batched together, it takes 13.5s to finish serving for an LLaMA2-13B HF-worker deployed on one NVIDIA A100 80GB GPU. If the requests with input lengths of 10 and 1024 are divided into two separate batches, the total serving time is only 7.6s. This is because all the requests are padded to 1024 tokens when they are batched



(a) Together batching. (b) Separate batching. (c) Serving time.

Fig. 8: Batching example. Under a given slice length of 128, various batching strategies have a large difference in the serving time. In sub-figure (a), white and blue grids represent pad and input tokens, respectively.

together, which greatly increases the per-iteration prefill and decoding latency for the requests with an input length of 10.

Accordingly, we propose the adaptive batcher which leverages a dynamic programming-based algorithm to partition requests into batches with the goal of minimizing the total batch serving time. By aiming to reduce serving time, the algorithm can make an excellent trade-off between mitigating padding and increasing batch size.

Pseudocode of the batching algorithm is presented in Algorithm 1. It first sorts requests in ascending order by their input lengths. Then the state array $\mathcal{T}$ is created, where $\mathcal{T}[i]$ denotes the minimal total batch serving time for the first i requests. The initial state $\mathcal{T}[0] = 0$ indicates that the total serving time is 0 when there is no request to batch. The position array $\mathcal{P}$ stores positions for partitioning requests into batches.

During dynamic programming, the algorithm sequentially traverses each request. For each request, the algorithm first treats it as a separate batch and then tries to batch it together with preceding requests to minimize the total batch serving time while guaranteeing that OOM errors will not occur. The state transfer function of dynamic programming is

$$\mathcal{T}[i] = \min_{0<j\leq i} (\mathcal{T}[j-1] + T_{serve}(i-j+1, L_i, S)), \quad (10)$$

where $i - j + 1$ computes the batch size. Since requests are sorted by the request input length at the beginning when traversing to the ith request, its input length is the maximal input length of the first i requests, and hence its input length can be utilized as the batch input length L_i to calculate the estimated batch serving time T_{serve}. After the dynamic programming, the requests are divided into batches according to the recorded splitting positions.

In the batching procedure, whether OOM errors will occur is determined by the memory usage estimator, and the batch serving time is estimated by the serving time estimator.

Algorithm 1: Serving Time-Oriented Batching

Input: $\mathcal{R}$: a list of requests; S: the slice lenth
Output: $\mathcal{L}$: a list of batches

Sort $\mathcal{R}$ by the request input length in ascending order
n ← the number of requests in $\mathcal{R}$
// States denoting total serving time
$\mathcal{T}$ ← array of zeroes[0..n]
// Positions for splitting batches
$\mathcal{P}$ ← array of zeroes[0..n]
for $i \leftarrow 1$ **to** n **do**
  $\mathcal{P}[i] \leftarrow i-1$ /* Separate the ith request into a single batch */
  $L_i \leftarrow$ input length of the request $\mathcal{R}[i-1]$
  $\mathcal{T}[i] \leftarrow \mathcal{T}[i-1] + T_{serve}(1, L_i, S)$
  $j \leftarrow i-1$
  while $j > 0$ ***and not*** $OOM(i-j+1, L_i, S)$ **do**
    $t \leftarrow \mathcal{T}[j-1] + T_{serve}(i-j+1, L_i, S)$
    if $t < \mathcal{T}[i]$ **then**
      $\mathcal{T}[i] \leftarrow t$ /* Update the total serving time */
      $\mathcal{P}[i] \leftarrow j-1$ /* Record the splitting position */
    $j \leftarrow j-1$
$\mathcal{L}$ ← empty list // Now, i is equal to n
while $i > 0$ **do**
  $p \leftarrow \mathcal{P}[i]$
  Batch $\mathcal{R}[p:i]$ together and append the batch to $\mathcal{L}$
  $i \leftarrow p$
return $\mathcal{L}$

E. Balanced Load-Oriented Offloading

The offloader achieves load balancing among multiple workers using a max-min-based algorithm. The load of a worker is defined as the time it takes to serve all the batches in its local queue, and the initial load of each worker is 0.

The offloader offloads batches to workers one by one. At each offload, it schedules the batch with the longest estimated serving time to the worker with minimal load and then updates the worker's load with

$$T_{load}(w) \leftarrow T_{load}(w) + T_{serve}(N, L_i, S), \quad (11)$$

where $T_{load}(w)$ represents the load of the least loaded worker w and $T_{serve}(N, L_i, S)$ denotes the estimated serving time of the batch to be offloaded, where N, L_i, and S are the batch size, batch input length, and slice length, respectively.

After each time a worker completes batch serving, SCLS subtracts the estimated serving time of the batch it just served from its load. This can prevent the estimation error of serving time from accumulating in the worker's load.

F. Adaptive Schedule Interval Update

SCLS periodically fetches requests from the request pool and hands them over to the adaptive batcher at a time interval of T. After each batch offloading, SCLS updates T by

$$T \leftarrow \max(\lambda \cdot \min_{w}(T_{load}(w)), \Gamma), \quad (12)$$

where Γ is a pre-defined minimal time interval, $T_{load}(w)$ is the load of worker w, and λ is a factor less than 1.

When the load of each worker is light, T decreases so that requests can be processed by workers in time without waiting for a long time in the request pool, thus reducing the request response time. When there are a lot of batches waiting in the local queue of each worker, T increases so that the adaptive batcher is highly likely to batch more requests together, thus increasing the throughput. Due to the serving time estimation

Algorithm 2: DeepSpeed-Inference OOM Judgment

Input: L_i:batch input length; S: slice length; N: batch size

Output: An expression that OOM will occur if true

```
L ← L_i + S /* L ≤ 2048 under the experimental settings */
if L > 1024 then
    return N > 12
else if L > 512 then
    return N > 22
else
    return N > 28 /* Total token number L ≤ 512 */
```

error, λ is introduced to prevent workers from being idle when the load is over-estimated, and Γ is introduced to avoid feeding too few requests to the adaptive batcher when the load is under-estimated.

V. Experimental Evaluation

A. Experiment Setup

Experiments are conducted in the production environment at Ant Group, where 8 LLaMA-13B LLM instances are deployed with 8 NVIDIA A100 80GB GPUs connected over NVLink. We use LLaMA2-13B in the experiments because LLaMA's model architecture is representative and has been adopted by many LLMs. Besides, 13B-parameter LLMs can meet the needs of most real-world application scenarios in the production environment. We take the request throughput, average and tail response time as metrics to evaluate the performance of SCLS, following existing work on LLM serving [4], [5], [14].

We implement SCLS, DS-worker and HF-worker with Python 3.8. For the serving time estimator, we use `curve_fit` function from the scipy library [15] to fit Eq. (3) and Eq. (4). For the memory usage estimator, ζ in Eq. (9) is set to 0.9 for HF. For DS, SCLS determines if OOM errors will occur by rules that are presented in Algorithm 2. To fetch more requests while keeping workers busy, SCLS dynamically adjusts the schedule time interval T by Eq. (12), where λ is set to 0.5, Γ is set to 6s and 3s for HF and DS according to their inference speed, respectively. Moreover, the slice length is set to 128 if not specified.

Requests from CodeFuse's traces are sent in the order they actually arrived for 10 minutes. Arrival times are generated using Poisson distribution with various request rates. We set both the maximal raw request input length and the maximal generation length limit to 1024, covering length range settings of existing studies [4], [5], [16]–[18].

We compare SCLS with SLS and ILS schedulers. Since HF only supports static batching, for HF, we only compare SCLS with SLS. For DS, we compare SCLS with SLS and Deepspeed-Fastgen, an ILS scheduler implemented on top of DS. We don't compare SCLS with vLLM [19] because vLLM does not support static batching, and hence SCLS can not achieve the same kernel-level optimization with vLLM, which makes it challenging to achieve a fair comparison from a pure scheduling perspective. Besides, Deepspeed-Fastgen has been shown to outperform vLLM's performance [6]. Therefore, we think it reasonable and convincing to compare SCLS with Deepspeed-Fastgen.

- **Sequence-level Scheduling (SLS)** [2], [3]: The scheduler offloads requests to workers using the round-robin policy. Workers serve received requests in an FCFS manner with a fixed batch size. Since different engines have various M_{engine}, for HF and DS workers, the fixed batch size is set to 16 and 12 to avoid OOM errors, respectively.
- **Iteration-level Scheduling (ILS)** [6]: Deepspeed-FastGen, the ILS scheduler built on top of DS, achieves a low latency with continuous batching [4], paged attention [5], and chunked-prefill [20] techniques. However, it utilizes the round-robin policy to offload requests and adopts a conservative memory management mechanism that limits the number of parallel-processing requests. Since Deepspeed-FastGen is also implemented based on DS, when SCLS is integrated with DS, SCLS and ILS have the same kernel-level optimization, so a fair comparison can be made from a pure scheduling perspective.

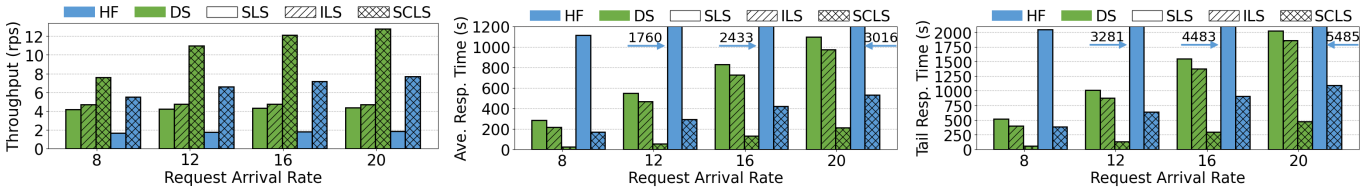
B. Overall Performance

We compare the performance of SCLS to the baseline schedulers in terms of request throughput and tail response time. The experimental results are depicted in Fig. 9, where different colors represent different inference engines, and various hatch styles represent various scheduling techniques. We can see that SCLS always outperforms SLS and ILS when integrating with various inference engines under different request arrival rates, as shown in Fig. 9.

For HF, SCLS can increase the throughput by 232.3% to 315.8%, decrease the average response time by 82.4% to 84.9%, and decrease the tail response time by 79.8% to 81.0% compared with SLS. For DS, SCLS increases the throughput by 82.5% to 191.9%, reduces the average response time by 80.8% to 91.1%, and decreases the tail response time by 76.6% to 88.9% compared with SLS.

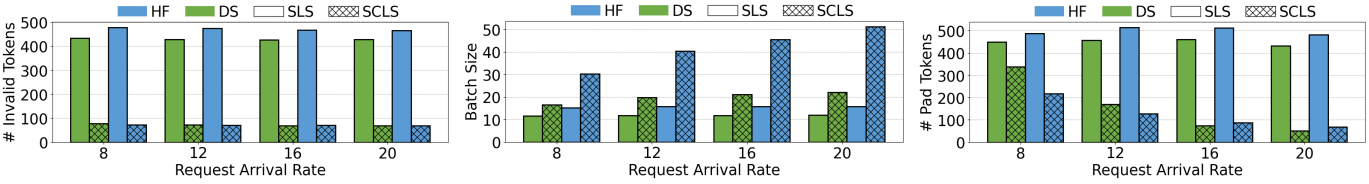
Compared with ILS, SCLS can support larger batch sizes without causing OOM errors, whereas Deepspeed-FastGen adopts a conservative memory management policy by limiting the number of parallel-processing requests. As a result, SCLS can improve the throughput by 61.6% to 171.0%, reduce the average request response time by 78.4% to 88.3%, and reduce the tail response time by 74.6% to 85.8% compared with ILS when integrated with DS.

Compared with SLS, the superiority of SCLS mainly comes from three sources. The first is to return completed requests in time, thus avoiding request waiting and greatly reducing the number of generated invalid tokens, as shown in Fig. 10a. The second is to serve requests with a large batch size to achieve high throughput, as shown in Fig. 10b. The third is to reduce the serving latency by mitigating the padding overhead using the adaptive batcher as shown in 10c. Due to the different memory management mechanisms of HF and DS, the maximal batch size that can be supported under the same batch input length and the same slice length is different, and therefore, SCLS obtains different performance gains when integrated with various inference engines.



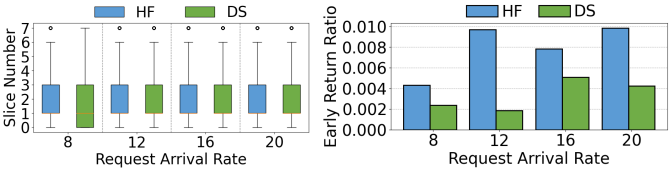
(a) Request throughput. (b) Average response time. (c) Tail (95%) response time.

Fig. 9: Request throughput, average response time and tail response time under various arrival rates.



(a) Average number of invalid tokens. (b) Average batch size. (c) Average number of pad tokens.

Fig. 10: Dive into the superiority of SCLS.



(a) Slice number distribution. (b) Early return ratio.

Fig. 11: Dive into the overhead of SCLS.

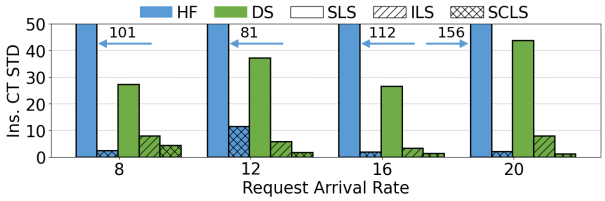


Fig. 12: Load imbalance under various request rates

C. SCLS Overhead

The overhead of SCLS comes from the re-computation of the prefill phase with each reschedule. We illustrate the distribution of request reschedule number (a.k.a. slice number) under different request rates in Fig. 11a. We find that the vast majority of requests have less than three reschedules. This is because their request generation lengths are small and can complete generation within three slices. Although the total serving time of requests with long generation lengths is increased due to multiple re-computations of the prefill phase, they also benefit from SCLS as their queuing time is greatly reduced due to increased throughput.

In Section IV-B, we mentioned that if all requests generate EOS before the generation length reaches the slice length, the requests will be returned early, which leads to inaccurate time estimation. To verify that this happens very rarely, in Fig. 11b, we calculate the proportion of these early returned batches to the total number of served batches. We can find that early return happens rarely, with a maximal proportion less than 1%. Hence, the early return has a negligible effect on the performance of SCLS.

D. Load Balance

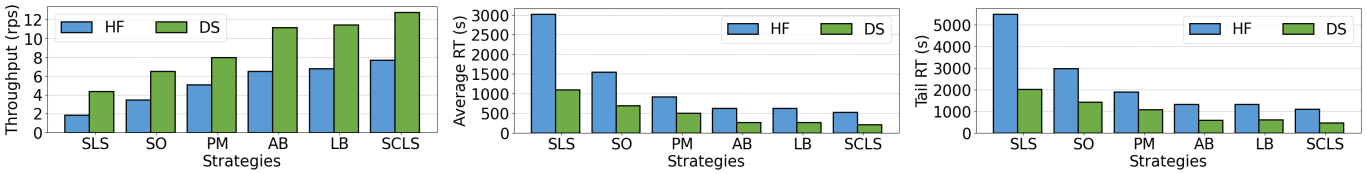
We calculated the standard deviation (STD) of each instance's completion time (CT) at the end of the experiment in Fig. 12. We find that the CT STD of SCLS is always the smallest, which indicates that the completion time of each instance is similar and SCLS achieves load balancing across multiple LLM instances. This is because the offloading algorithm can accurately estimate the load of workers and leverage the max-min algorithm to reduce the load gap between workers.

Without knowing the request generation length, SLS and ILS leverage the round-robin policy to offload requests, which may schedule requests with long generation lengths and requests with short generation lengths to different instances in a period of time. Such an unbalanced workload accumulates over time and leads to different instance completion times, Therefore, they have a higher standard deviation of instance completion time than SCLS.

E. Ablation Studies

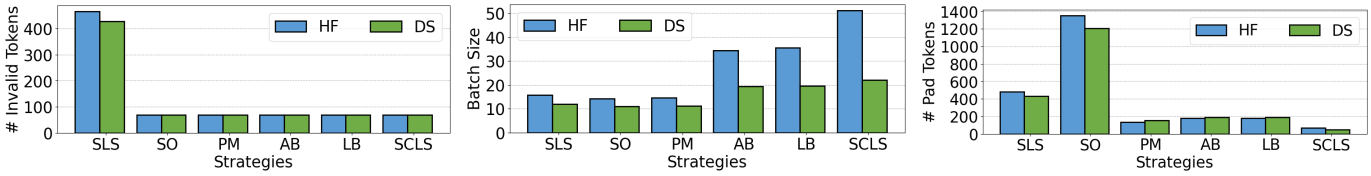
We demonstrate the performance gain of the design features of SCLS. We keep adding features to SLS to constitute multiple strategies and then verify their performance in terms of request throughput and response time at a request arrival rate of 20. The experimental results are presented in Fig. 13.

Slice-Only (SO) limits the number of iterations for static batching to the slice length, returning completed requests in time, and re-sent unfinished requests to workers. It offloads requests to workers using the round-robin policy. Workers serve received requests in an FCFS manner with a fixed batch size. **Padding-Mitigating (PM)** implements an incomplete batching algorithm on top of SO, where the batch size is limited to 12 and 16 for DS and HF, respectively. Besides, PM fetches requests from the request pool at the fixed time interval Γ and offloads batches to workers with the round-robin policy as well. **Adaptive-Batching (AB)** lifts the fixed batch size limitation on the basis of PM and implements the complete serving time-oriented batching algorithm. **Load-Balancing (LB)** adds the max-min offloading policy to AB to achieve a balanced workload across multiple LLM instances. By integrating the adaptive batch schedule update mechanism on LB, SCLS is constructed.



(a) Request throughput. (b) Average response time (RT). (c) Tail (95%) response time (RT).

Fig. 13: Request throughput, average response time and tail response time under various strategies.



(a) Average number of invalid tokens. (b) Average batch size. (c) Average number of pad tokens.

Fig. 14: Dive into the results of ablation experiments.

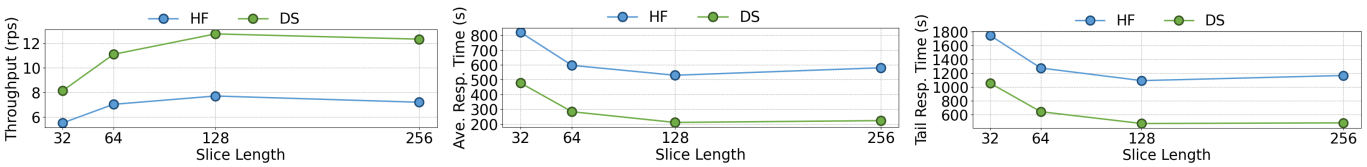
As shown in Fig. 13, the performance of strategies gradually enhances as more features are integrated, where the throughput increases and the response time decreases. Compared with SLS, SO is able to return completed requests timely, thus gaining a performance boost. Compared with SO, PM continues to improve performance by reducing the additional computational overhead caused by pad tokens using the incomplete batching algorithm. Compared with PM, AB increases the batch size to improve throughput as shown in Fig.14b. LB achieves a balanced load across workers, thus preventing workers from being idle and fully utilizing workers' computing capability to further improve the throughput. The adaptive schedule interval update mechanism allows more requests to be fetched when the workload is heavy, giving the batching algorithm more chances to increase the batch size and reduce pad tokens.

We also present the average invalid token number of requests, the average batch size, and the average pad token number of requests for these strategies in Fig. 14. We can see that generation slicing can significantly reduce invalid tokens, and the batching algorithm can greatly reduce pad tokens and increase the batch size. Although the number of pad tokens of AB and LB is slightly higher than that of PM, the performance of AB and LB is higher than that of PM due to the great increase in batch size, which confirms that by aiming to reduce serving time, the batching algorithm can make an excellent trade-off between mitigating padding and increasing batch size.

F. Impact of Slice Length

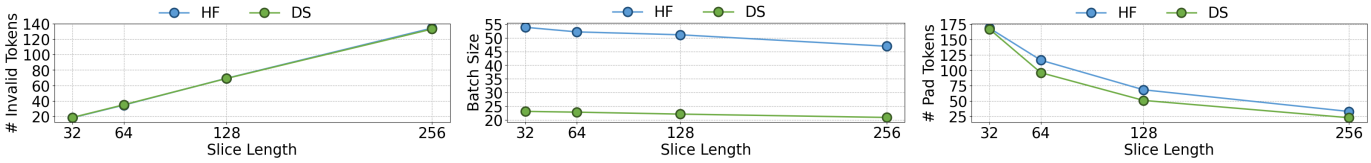
In this section, we explore the impact of slice length on the performance of SCLS. We evaluate the performance of SCLS at a request rate of 20 under various slice lengths. The experimental results are shown in Fig. 15. Besides, we also present the average invalid token number of requests, the average batch size, and the average pad token number of requests in Fig. 16. Mover, we also depict the slice number distribution and early return ratio in Fig. 17 to analyze how the overhead varies with the slice length.

From Fig. 15, we can find that with the increase in slice length, the performance first increases and then decreases. This is because when the slice length is set too small, there are a lot of reschedules of requests. The reschedules cause requests to be padded multiple times, and hence the computational overhead caused by padding is severe when the slice length is set small, as shown in Fig. 16c. Besides, the frequent reschedules cause a large number of prefill re-computations. Although with a small slice length, completed requests can be returned in a timely manner, and the batch size can be relatively large, it is not enough to make up for the computational overhead of the large amount of re-padding and prefill re-computations. When the slice length is increased, although the batch size decreases slightly, the number of reschedules decreases dramatically as shown in Fig. 17a, and hence the performance is improved. However, setting the slice length too long not only further decreases the batch size as shown in Fig. 16b, but also causes more serious request waiting, and more invalid tokens are generated as shown in Fig.16a, so the performance decreases. Furthermore, a long slice length causes a large early return ratio as shown in Fig. 17b, which makes the serving time estimation inaccurate, and hence the workload becomes unbalanced as shown in Fig. 18. Therefore, when setting the slice length, we need to do a trade-off between the request padding, the batch size, the prefill re-computations, and the request waiting.



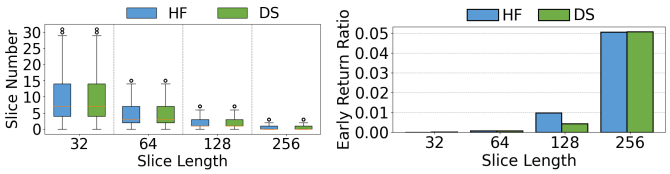
(a) Request throughput. (b) Average response time. (c) Tail (95%) response time.

Fig. 15: Request throughput, average response time and tail response time under various slice lengths.



(a) Average number of invalid tokens. (b) Average batch size. (c) Average number of pad tokens.

Fig. 16: Dive into the slice length impact on SCLS performance .



(a) Slice number distribution. (b) Early return ratio.

Fig. 17: Dive into the slice length impact on SCLS overhead.

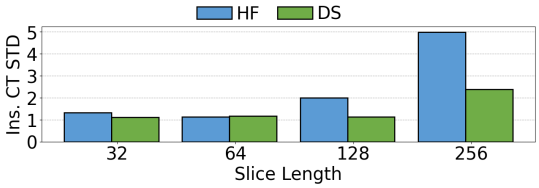


Fig. 18: Load imbalance under various slice lengths

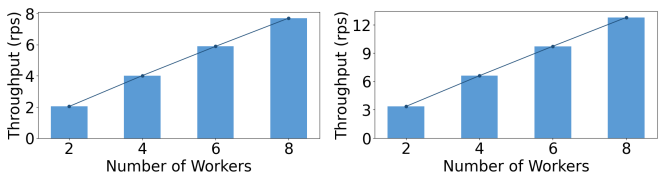
Since the performance of SCLS monotonically increases first and then monotonically decreases with the increase of the slice length, for application scenarios with specific request input and output distribution, we can simply use the binary search to quickly find the optimal setting of slice length with a time complexity of $\log(n)$, where n is the possible maximal slice length, making the search for the optimal slice length simple and efficient.

G. Handling Long-Generation Requests

We increase the proportion of requests with generation length greater than 512 to 60% and re-conduct the experiments at a request rate of 8. Since ILS has been demonstrated to be consistently superior to SLS, we only compare SCLS with ILS using DS as the inference engine. The results show that SCLS can improve the request throughput by 117% compared with the ILS scheduler. Besides, the average response time is reduced by 15.7% when the slice length increases from 128 to 256 due to greatly decreased re-prefill, which confirms that SCLS can effectively handle long generation scenarios by properly increasing the slice length.

H. Scalability

We evaluate the throughput of SCLS under different numbers of workers at a request arrival rate of 20, where each worker is deployed on an NVIDIA A100 80GB GPU. The experimental results are shown in Fig. 19. We can see that the throughput grows linearly with the number of workers, which confirms the scalability of SCLS.



(a) Huggingface-Transformers. (b) Deepspeed-Inference.

Fig. 19: Scalability of SCLS.

VI. Related Works

A. Model Compression

. Model quantization [21]–[25] converts the numerical weight precision of LLMs into fewer bits to reduce the memory usage of LLM parameters. The technique of pruning [26]–[29] eliminates redundant components of LLMs to increase the inference speed. These approaches can affect LLM accuracy while SCLS is a request scheduling framework that does not make any modification to LLM parameters and does not affect LLM accuracy. Besides, since SCLS is an inter-instance request scheduling framework, these works are orthogonal to SCLS, and SCLS can also improve the serving efficiency for quantized and pruned LLMs.

B. Low-level Optimization.

Kernel optimization techniques such as flash attention [30], [31] and paged attention [5] speed up LLM inference by implementing efficient CUDA kernels to improve computational efficiency. Parallelism optimization [32]–[36] proposes to split the model at intra-layer and inter-layer granularity to enable distributed LLM inference. These studies are always employed to make a single LLM instance more efficient in inference. However, SCLS is a scheduling system that schedules requests to multiple workers. It leverages proposed adaptive batching and scheduling algorithms to fully use the inference engine and balance load among multiple workers. Moreover, these studies are orthogonal to SCLS as well. SCLS can be augmented with these low-level techniques.

C. LLM Serving Systems.

Existing serving systems [2], [3] adopt the SLS to serve requests in an FCFS manner with a fixed batch size. Orca [4] and Deepspeed-FastGen [6] leverage the ILS to improve the serving efficiency, but they adopt conservative memory management strategies that limit throughput. PiA [16] and S^3 [17] propose to roughly predict the request generation length to enhance serving efficiency. However, they require significant computational overhead to fine-tune LLMs or train predictors. Recent studies [14], [18], [20], [37]–[45] propose a series of advanced techniques such as chunked prefill, disaggregated inference, and multi-lora serving. These works are designed for continuous batching, and we will combine these techniques with generation slicing in the future.

VII. Generation Slicing for Continuous Batching

For continuous batching, if the generation length of a request is too long, it will consume a large amount of memory, which causes newly arrived requests to queue up and can not be added to the serving process due to insufficient memory. By limiting the number of generated tokens for requests in each schedule to a small slice length, we can solve this problem by timely re-scheduling the long request to an instance with more free memory. Besides, we can also accurately estimate the memory consumed by requests in each schedule to serve as many requests in parallel as possible without causing OOM errors or request preemption and achieve balanced memory consumption across multiple LLM instances. Besides, for requests re-scheduled to the same LLM instance, we can mitigate the prefill re-computation overhead by reusing the previously produced key-value cache with prefix-caching [46] or proactively swapping the key-value cache between GPU and CPU memories. For requests re-scheduled to different LLM instances, the prefill re-computation can be amortized with decoding using chunked prefill [20].

VIII. Conclusion and Future Works

In this paper, we propose the core idea of generation slicing and design slice-level scheduling (SCLS) to achieve high-throughput and load-balanced LLM serving with static batching. By splitting the predefined maximal generation length limit into slices, SCLS can provide a precise range of serving time and memory usage for batches, laying the foundation for the design of adaptive bathing and max-min offloading algorithms. Extensive experiments are conducted to confirm the superiority of SCLS in throughput improvement and load balance. In the future, we will apply generation slicing to continuous batching and mitigate the overhead of the prefill re-computation by amortizing prefill computation using the chunked prefill technique and proactively swapping the key-value cache between the CPU and GPU memory.

References

[1] "Codefuse," https://codefuse.alipay.com, 2024.

[2] "Triton inference server," https://github.com/triton-inference-server/server, 2023.

[3] "Tensorflow serving," https://github.com/tensorflow/serving, 2023.

[4] G.-I. Yu, J. S. Jeong, G.-W. Kim *et al.*, "Orca: A distributed serving system for transformer-based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.

[5] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.

[6] "Deepspeed-fastgen," https://github.com/microsoft/DeepSpeed-MII, 2024.

[7] B. Radunovic and J.-Y. Le Boudec, "A unified framework for max-min and min-max fairness with applications," *IEEE/ACM Transactions on networking*, vol. 15, no. 5, pp. 1073–1083, 2007.

[8] "Chatgpt," https://chat.openai.com, 2024.

[9] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan *et al.*, "Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–15.

[10] H. Touvron, L. Martin, K. Stone *et al.*, "Llama 2: Open foundation and fine-tuned chat models," 2023.

[11] "Sharegpt," https://sharegpt.com, 2024.

[12] T. Wolf, L. Debut, V. Sanh *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.

[13] A. Paszke, S. Gross, F. Massa *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.

[14] B. Wu, R. Zhu, Z. Zhang, P. Sun, X. Liu, and X. Jin, "dlora: Dynamically orchestrating requests and adapters for lora llm serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 911–927.

[15] P. Virtanen, R. Gommers, T. Oliphant *et al.*, "Fundamental algorithms for scientific computing in python and scipy 1.0 contributors. scipy 1.0," *Nat. Methods*, vol. 17, pp. 261–272, 2020.

[16] Z. Zheng, X. Ren, F. Xue *et al.*, "Response length perception and sequence scheduling: An llm-empowered llm inference pipeline," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[17] Y. Jin, C.-F. Wu, D. Brooks, and G.-Y. Wei, "s^3: Increasing gpu utilization during generative inference for higher throughput," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[18] Y. Sheng, S. Cao, D. Li, B. Zhu, Z. Li, D. Zhuo, J. E. Gonzalez, and I. Stoica, "Fairness in serving large language models," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 965–988.

[19] "vllm," https://github.com/vllm-project/vllm, 2024.

[20] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 117–134.

[21] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "OPTQ: Accurate quantization for generative pre-trained transformers," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=tcbBPnfwxS

[22] G. Xiao, J. Lin, M. Seznec *et al.*, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.

[23] W. Shao, M. Chen, Z. Zhang *et al.*, "Omniquant: Omnidirectionally calibrated quantization for large language models," in *The Twelfth International Conference on Learning Representations*, 2023.

[24] J. Chee, Y. Cai, V. Kuleshov *et al.*, "Quip: 2-bit quantization of large language models with guarantees," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[25] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 87–100, 2024.

[26] R. Xu, F. Luo, and C. o. Wang, "From dense to sparse: Contrastive pruning for better pre-trained language model compression," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 10, 2022, pp. 11 547–11 555.

[27] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, "A simple and effective pruning approach for large language models," in *The Twelfth International Conference on Learning Representations*, 2023.

[28] X. Ma, G. Fang, and X. Wang, "Llm-pruner: On the structural pruning of large language models," *Advances in neural information processing systems*, vol. 36, pp. 21 702–21 720, 2023.

[29] H. Shao, B. Liu, and Y. Qian, "One-shot sensitivity-aware mixed sparsity pruning for large language models," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 11 296–11 300.

[30] T. Dao, D. Fu, S. Ermon *et al.*, "Flashattention: Fast and memory-efficient exact attention with io-awareness," *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.

[31] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," in *The Twelfth International Conference on Learning Representations*, 2023.

[32] Y. Huang, Y. Cheng, A. Bapna *et al.*, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," *Advances in neural information processing systems*, vol. 32, 2019.

[33] D. Narayanan, A. Harlap, A. Phanishayee *et al.*, "Pipedream: generalized pipeline parallelism for dnn training," in *Proceedings of the 27th ACM symposium on operating systems principles*, 2019, pp. 1–15.

[34] D. Narayanan, M. Shoeybi, J. Casper *et al.*, "Efficient large-scale language model training on gpu clusters using megatron-lm," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–15.

[35] X. Miao, Y. Wang, Y. Jiang *et al.*, "Galvatron: Efficient transformer training over multiple gpus using automatic parallelism," *Proceedings of the VLDB Endowment*, vol. 16, no. 3, pp. 470–479, 2022.

[36] X. Miao, Y. Shi, Z. Yang, B. Cui, and Z. Jia, "Sdpipe: A semi-decentralized framework for heterogeneity-aware pipeline-parallel training," *Proceedings of the VLDB Endowment*, vol. 16, no. 9, pp. 2354–2363, 2023.

[37] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci, "Atom: Low-bit quantization for efficient and accurate llm serving," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 196–209, 2024.

[38] Z. Liu, J. Wang, T. Dao, T. Zhou, B. Yuan, Z. Song, A. Shrivastava, C. Zhang, Y. Tian, C. Re *et al.*, "Deja vu: Contextual sparsity for efficient llms at inference time," in *International Conference on Machine Learning*. PMLR, 2023, pp. 22 137–22 176.

[39] J. Fang, Y. Yu, C. Zhao, and J. Zhou, "Turbotransformers: an efficient gpu serving system for transformer models," in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2021, pp. 389–402.

[40] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "Flexgen: High-throughput generative inference of large language models with a single gpu," in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 094–31 116.

[41] X. Miao, C. Shi, J. Duan, X. Xi, D. Lin, B. Cui, and Z. Jia, "Spotserve: Serving generative large language models on preemptible instances," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 1112–1127.

[42] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 193–210.

[43] Y. Fu, L. Xue, Y. Huang, A.-O. Brabete, D. Ustiugov, Y. Patel, and L. Mai, "Serverlessllm:low-latency serverless inference for large language models," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 135–153.

[44] Y. Sheng, S. Cao, D. Li, C. Hooper, N. Lee, S. Yang, C. Chou, B. Zhu, L. Zheng, K. Keutzer *et al.*, "Slora: Scalable serving of thousands of lora adapters," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 296–311, 2024.

[45] R. Abhyankar, Z. He, V. Srivatsa, H. Zhang, and Y. Zhang, "Infercept: Efficient intercept support for augmented large language model inference," in *Forty-first International Conference on Machine Learning*.

[46] C. Lin, Z. Han, C. Zhang, Y. Yang, F. Yang, C. Chen, and L. Qiu, "Parrot: Efficient serving of llm-based applications with semantic variable," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 929–945.